

Клас и обекти

Класът в C++/Java/Python е градивният елемент, който води до ООП. Това е дефиниран от потребителя тип данни, който има член данни и член функции. Член данните са променливите на данните, а член функциите, се използват за манипулиране на тези променливи. Класът в Python се дефинира с помощта на ключовата дума **class**, последвана **от името на класа**.

Обектът е екземпляр на клас. Когато е дефиниран клас, не се разпределя памет, но когато се инстанцира (т.е. е създава се обект), тогава се разпределя памет.

Функцията **init ()** в класа се изпълнява автоматично всеки път, при създаване на нов обект, и съответства на конструктора в обектно-ориентираната парадигма. Тази функция се използва за инициализиране на членовете на данни на класа. В повечето от езиците за ООП, спецификаторите за достъп се използват за ограничаване на достъпа до променливите и функциите на клас. Езиците, които са ОО използват три типа спецификатори за достъп: частен, публичен и защитен. В Python всички променливи и функции-членове на клас са публични по подразбиране. Добавянето на префикс (двойно долно подчертаване) към член променлива или функция ги прави частни.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Пример:

Да се намери евклидовото разстояние между две точки в триизмерно пространство **с**, като се създаде клас `point` и обекти.

Да се създаде конструктор (`init()`) и две функции `distancefromorigin()` и `distance()`.

Разстояние е величина, използвана в математика и физика като мярка за това колко близко или колко далече е едно тяло от друго.

Разстояние между две точки

Разстоянието между две точки **A** и **B** в пространството (декартови координати) се дава с:

$$d_{AB} = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad \text{където} \quad A = \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix} \in \mathbb{R}^n \quad B = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} \in \mathbb{R}^n$$

за тримерно пространство R^3 формулата добива вида: $d_{AB} = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2 + (z_B - z_A)^2}$

или опростено, ако са дадени 2 точки с координати (x_1, y_1, z_1) и (x_2, y_2, z_2) разстоянието между тях е:

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}.$$

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Решение:

1) Задаваме т.А с координати (1, 2, 3) и т.В с координати (2, 3, 4)

т.е. coordinates of a first point P1(x1, y1, z1): 1 2 3

и

coordinates of a second point P2(x2, y2, z2): 2 3 4

2) Прилагаме формулата: $d = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$.

$$\sqrt{x_1^2 + y_1^2 + z_1^2} = \sqrt{1*1 + 2*2 + 3*3} = \sqrt{1+4+9} = \sqrt{14} = 3.7416573867739413$$

$$\sqrt{x_2^2 + y_2^2 + z_2^2} = \sqrt{2*2 + 3*3 + 4*4} = \sqrt{4+9+16} = \sqrt{29} = 5.385164807134504$$

$$\sqrt{x_2^2 + y_2^2 + z_2^2} = \sqrt{1*1 + 1*1 + 1*1} = \sqrt{3} = 1.7320508075688772$$

Distance from origin to P1: 3.7416573867739413

Distance from origin to P2: 5.385164807134504

Distance from P1 to P2: 1.7320508075688772

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Пример:

Да се намери евклидовото разстояние между две точки в триизмерно пространство **c**, като се създаде клас **point** и обекти. Да се създаде конструктор (**init()**) и две функции **distancefromorigin()** и **distance()**.

```
# To find the Euclidean distance between two points in a three dimensional space using class and objects.
# Class called point consists of a constructor (init()) and two functions distancefromorigin() and distance()
import math
class point():
    def __init__(self,a,b,c):
        self.x=a
        self.y=b
        self.z=c
    def distancefromorigin(self):
        return ((self.x ** 2) + (self.y ** 2) +(self.z ** 2)) ** 0.5
    def distance(self, point2):
        xdiff = self.x-point2.x
        ydiff = self.y-point2.y
        zdiff = self.z-point2.z
        dist = math.sqrt (xdiff**2 + ydiff**2+ zdiff**2)
        return dist
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
x1,y1,z1= (input("Enter the coordinates of a first point P1(x1,y1,z1): ")).split()
x1,y1,z1 =[int(x1),int(y1), int(z1)]
x2,y2,z2= (input("Enter the coordinates of a second point P2(x2,y2,z2): ")).split()
x2,y2,z2 =[int(x2),int(y2),int(z2)]
# P1 and P2 are objects of point class
p1 = point(x1,y1,z1)
p2 = point(x2,y2,z2)
print('Distance from origin to P1:', p1.distancefromorigin())
print('Distance from origin to P2:', p2.distancefromorigin())
print('Distance from P1 to P2:',p1.distance(p2))
```

Резултат:

Enter the coordinates of a first point P1(x1,y1,z1): 1 2 3

Enter the coordinates of a second point P2(x2,y2,z2): 2 3 4

Distance from origin to P1: 3.7416573867739413

Distance from origin to P2: 5.385164807134504

Distance from P1 to P2: 1.7320508075688772

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

main.py		Shell
<pre>1 import math 2 class point(): 3 def __init__(self,a,b,c): 4 self.x=a 5 self.y=b 6 self.z=c 7 def distancefromorigin(self): 8 return ((self.x ** 2) + (self.y ** 2) + (self.z ** 2)) ** 0.5 9 def distance(self, point2): 10 xdiff = self.x-point2.x 11 ydiff = self.y-point2.y 12 zdiff = self.z-point2.z 13 dist = math.sqrt (xdiff**2 + ydiff**2+ zdiff**2) 14 return dist 15 16 x1,y1,z1= (input("Enter the coordinates of a first point P1(x1,y1,z1): " 17)).split() 18 x1,y1,z1 =[int(x1),int(y1), int(z1)] 19 x2,y2,z2= (input("Enter the coordinates of a second point P2(x2,y2,z2): " 20)).split() 21 x2,y2,z2 =[int(x2),int(y2),int(z2)] 22 # P1 and P2 are objects of point class 23 p1 = point(x1,y1,z1) 24 p2 = point(x2,y2,z2) 25 print('Distance from origin to P1:', p1.distancefromorigin()) 26 print('Distance from origin to P2:', p2.distancefromorigin())</pre>		<pre>Enter the coordinates of a first point P1(x1,y1,z1): 1 2 3 Enter the coordinates of a second point P2(x2,y2,z2): 2 3 4 Distance from origin to P1: 3.7416573867739413 Distance from origin to P2: 5.385164807134504 Distance from P1 to P2: 1.7320508075688772 ></pre>

Activate
Go to DC sat

КЛАС

- Класът има атрибути и методи.
- Класът е като „конструктор на обекти“ или „план“ за създаване на обекти.
- Ключовата дума за създаване на класа е “class”

Синтаксис

```
class class_name:  
statement(s);
```

ОБЕКТ

- Обектът е екземпляр на класа.
- Един клас може да има произволен брой обекти.
- Обектите могат да предават съобщение по между си.

Синтаксис

```
object_name = class_name(parameters);
```

АТРИБУТИ

- Атрибутите са идентификаторите, които могат да съхраняват данни.
- Тези атрибути се използват в методите на класа.
- Един клас може да има произволен брой атрибути.

МЕТОДИ

- Методите не са нищо друго освен функции.
- Методите приемат параметри като вход и, ако е необходимо, връщат изход.
- Методите могат да бъдат идентифицирани чрез скоби.

Пример:

Създайте клас с име Person, използвайте функцията init (), за да зададете стойности за име и възраст. Нека метода init () да се извиква автоматично всеки път, когато класът създава нов обект.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
k = Person("John", 36)
print(k.name)
print(k.age)
```

Резултат:
John
36

main.py	  	Shell
<pre>1 class Person: 2 def __init__(self, name, age): 3 self.name = name 4 self.age = age 5 k = Person("John", 36) 6 print(k.name) 7 print(k.age)</pre>		<pre>John 36 ></pre>

Инициализация с използване на SELF

Ключовата дума `self` в Python се използва за всички екземпляри в даден клас. Използвайки ключовата дума `self`, така може лесно да се получи достъп до всички екземпляри, дефинирани в рамките на даден клас, включително неговите методи и атрибути.

init

init е един от запазените методи в Python. В ООП той е известен като конструктор. Методът init може да бъде извикан, когато обект е създаден от класа и е необходим достъп за инициализиране на атрибутите на класа.

ДЕФИНИЦИЯ НА КЛАСА И НАСЛЕДЯВАНЕ СЪЗДАВАНЕ НА КЛАСОВЕ

Инструкцията за клас създава нова дефиниция на клас. Името на класа непосредствено следва клас ключови думи, последван от двоеточие, както следва:

```
class ClassName:  
'Optional class documentation string'  
class_suite
```

- Класът има документационен низ, който може да бъде достъпен чрез `ClassName.doc`.
- `class_suite` се състои от всички съставни изрази, дефиниращи членове на класа, атрибути на данни и функции.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Пример:

Класът на Python.

```
class Employee:
    empCount = 0
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.empCount += 1
    def displayCount(self):
        print ("Total Employee %d" % Employee.empCount)
    def displayEmployee(self):
        print ("Name : ", self.name, ", Salary: ", self.salary)

emp1 = Employee("Zara",2000)
emp1.displayEmployee()
emp2 = Employee("Manni", 5000)
emp2.displayEmployee()
emp2.displayCount()
```

Резултат:

```
Name : Zara , Salary: 2000
Name : Manni , Salary: 5000
Total Employee 2
```

```
class Employee:
    empCount = 0
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.empCount += 1
    def displayCount(self):
        print ("Total Employee %d" % Employee.empCount)
    def displayEmployee(self):
        print ("Name : ", self.name, ", Salary: ", self.salary)

emp1 = Employee("Zara", 2000)
emp1.displayEmployee()
emp2 = Employee("Manni", 5000)
emp2.displayEmployee()
emp2.displayCount()
```

```
Name : Zara , Salary: 2000
Name : Manni , Salary: 5000
Total Employee 2
>
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

- Променливата `empCount` е променлива на класа, чиято стойност се споделя между всички екземпляри на този клас. Това може да бъде достъпно като `Employee.empCount` от вътре в класа или извън класа.
- Първият метод `init()` е специален метод, който се нарича конструктор на клас или метод за инициализация, който Python извиква, когато създавате нов екземпляр на този клас.
- Когато се декларират методи на класа те се декларират, като нормални функции с изключение, на **първият аргумент на всеки метод, който е `self`**. Python добавя аргумента `self` към списъка вместо вас; не е необходимо да го включвате, когато извиквате методите.

МОДИФИКАТОРИ ЗА ДОСТЪП В PYTHON - PUBLIC, PRIVATE И PROTECTED

Класическите обектно-ориентирани езици, като C++ и Java, контролират достъпа до ресурсите на класа чрез публични, частни и защитени ключови думи.

Частните членове на клас имат отказан достъп от средата извън класа. С тях може да се работи само в рамките на класа.

Публичните членове (обикновено методи, декларирани в клас) са достъпни извън класа. Обектът от същия клас е необходим за извикване на публичен метод. Това подреждане на променливи на частния екземпляр и публични методи осигурява принцип на капсулиране на данни.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Защитените членове на клас са достъпни в рамките на класа и също така са достъпни за неговите подкласове. Някоя друга среда няма разрешен достъп до него. Това позволява специфични ресурси на родителския клас да бъдат наследени от дъщерния клас.

Python няма механизъм, който ефективно да ограничава достъпа до която и да е променлива или метод на екземпляр. Python предписва конвенция за префикс на името на променливата/метода с единично или двойно подчертаване, за да се емулира поведението на защитени и частни спецификатори за достъп.

Всички членове в клас на Python са публични по подразбиране. Всеки член може да бъде достъпен извън средата на класа.

СЪЗДАВАНЕ НА ИНСТАНЦИОННИ ОБЕКТИ

За да създадете екземпляри на клас, вие извиквате класа, използвайки име на класа и предавате всички аргументи, които неговият метод `init` приема.

"This would create first object of Employee class,,

```
emp1 = Employee("Zara", 2000)
```

"This would create second object of Employee class"

```
emp2 = Employee("Manni", 5000)
```

Accessing Attributes

```
emp1.displayEmployee()
```

```
emp2.displayEmployee()
```

Можете да получите достъп до атрибутите на обекта, като използвате оператора точка с обект. Променливата на класа ще бъде достъпна чрез името на класа, както следва.

```
print ("Total Employee %d" % Employee.empCount)
```

НАСЛЕДЯВАНЕ (INHERITANCE):

Наследяването е механизъм, при който един клас (производен клас) придобива свойството на друг клас (базов клас). С наследяване можем да използваме повторно променливите и методите на съществуващия клас. Съществуващият клас се нарича базов клас, а новият клас се нарича производен клас. Следователно, наследяването улеснява повторната употреба и е важна концепция за ООП.

Видовете наследяване са: единично наследяване, множествено наследяване, многостепенно наследяване, йерархично наследяване и хибридно наследяване.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Единичното наследяване позволява на произведен клас да използва променливите и функциите, дефинирани в съществуващ клас. При множествено наследяване извлеченият клас наследява характеристиките от повече от един съществуващ клас.

В Python синтаксисът за дефиниране на единично наследяване е клас $z(x)$, където x е името на базовия клас, а z е името на извлечения клас. Аналогично, множественото наследяване се дефинира с помощта на синтактичния клас $z(x, y)$, където x и y са имената на базовите класове, а z е името на извлечения клас.

Прелимства на наслежаването (Advantages of inheritance)

Наследяването е способността на един клас да извлича или наследява свойствата от друг клас. Ползите от наследяването са:

- Представява добре отношенията в реалния свят.
- Осигурява повторна употреба на код. Не е нужно да пишем един и същ код отново и отново. Освен това ни позволява да добавяме повече функции към клас, без да го променяме.
- Той има преходен характер, което означава, че ако клас B наследява от друг клас A , тогава всички подкласове на B автоматично ще се наследят от клас A .

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Пример:

```
class person:
    def __init__(self,n,a):
        self.name = n
        self.age = a
    # employee is the class derived
class employee(person):
    def __init__(self, n,a, d, s):
        person.__init__(self, n,a)
        self.designation = d
        self.salary = s

    def show(self):
        print("Employee Details:")
        print(" Name: ", self.name,"\n Age:", self.age, "\n Designation:", self
        .designation, "\n Salary:", self.salary)

e = person("Arun",35)
r = employee("John", 30, "admin", 5000)
r.show()
e.show() # ще изведе грешка
```

Employee Details:
Name: John
Age: 30
Designation: admin
Salary: 5000
Traceback (most recent call last):
File "<string>", line 20, in <module>
AttributeError: 'person' object has no attribute 'show'
> #

Допълваме примера с класове – student и resident

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
# student is a base class
class student:
    def __init__(self, id, rno):
        self.studentId = id
        self.roomno = rno

# resident is a class derived from person and student using multiple inheritance
class resident(person, student):
    def __init__(self, n, a, id,rno):
        person.__init__(self, n, a)
        student.__init__(self, id, rno)

    def show(self):
        print("Resident Details:")
        print(" Name:", self.name, "\n Age: ",self.age, "\n Id:", self.studentId, "\n Room
no.:", self.roomno)
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Creating objects of employee and resident classes

e = employee("Arun",35,"Data analyst",50000)

r = resident("John", 30, 201900025,203)

e.show()

r.show()

Резултат:

Employee Details:

Name: Arun

Age: 35

Designation: Data analyst

Salary: 50000

Resident Details:

Name: John

Age: 30

Id: 201900025

Room no.: 203

>

```
main.py
1~ class person:
2~ def __init__(self,n,a):
3~     self.name = n
4~     self.age = a
5~     # employee is the class derived from person using single inheritance
6~
7~ class employee(person):
8~     def __init__(self, n, a, d, s):
9~         person.__init__(self, n, a)
10~         self.designation = d
11~         self.salary = s
12~
13~     def show(self):
14~         print("Employee Details:")
15~         print(" Name: ", self.name, "\n Age:", self.age, "\n Designation:", self
            .designation, "\n Salary:", self.salary)
16~ # student is a base class
17~ class student:
18~     def __init__(self, id, rno):
19~         self.studentId = id
20~         self.roomno = rno
21~ # resident is a class derived from person and student using multiple inheritance
22~ class resident(person, student):
23~     def __init__(self, n, a, id,rno):
24~         person.__init__(self, n, a)
```

Shell

Employee Details:
Name: Arun
Age:35
Designation:Data analyst
Salary:50000
Resident Details:
Name: John
Age: 30
Id:201900025
Room no.:203
> |

OPERATOR OVERLOADING

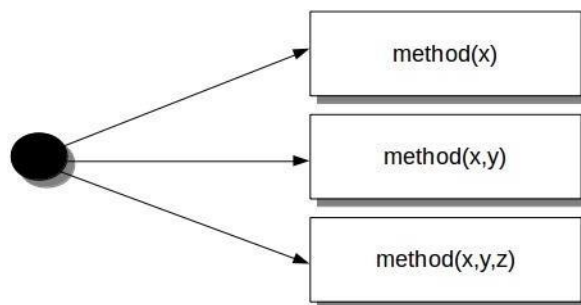
OVERLOADING оператора е важна концепция в ООП. Това е вид полиморфизъм, при който дефинирано от потребителя значение може да бъде дадено на оператор в допълнение към предварително дефинираното значение за оператора.

Претоварването на оператора ни позволява да предефинираме начина, по който операторът работи за дефинирани от потребителя типове, като обекти. Не може да се използва за вградени типове като `int`, `float`, `char` и т.н., Например операторът `'+'` може да бъде претоварен, за да извърши добавяне на два обекта от клас на разстояние.

Python предоставя някаква специална функция или магическа функция, която се извиква автоматично, когато е свързана с този конкретен оператор. Например, когато използваме `+` оператор за обекти, автоматично се извиква магическият метод `add()`, в който значението/операцията за `+` оператор се дефинира за дефинирани от потребителя обекти.

METHOD OVERLOADING

В Python можете да дефинирате метод по такъв начин, че да има няколко начина да го извикате. В зависимост от дефиницията на функцията, тя може да бъде извикана с нула, един, два или повече параметри.



Фиг.1

Пример:

Да се създаде клас distance за измерване на разстоянието във футове и инчове.

distance is a class. Distance is measured in terms of feet and inches

class distance:

```
def __init__(self,f,i):
```

```
    self.feet=f
```

```
    self.inches=i
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

overloading of binary operator > to compare two distances

```
def __gt__(self,d):  
    if(self.feet>d.feet):  
        return(True)  
    elif((self.feet==d.feet) and (self.inches>d.inches)):  
        return(True)  
    else:  
        return(False)
```

overloading of binary operator + to add two distances

```
def __add__(self, d):  
    i=self.inches + d.inches  
    f=self.feet + d.feet  
    if(i>=12):  
        i=i-12  
        f=f+1  
    return distance(f,i)
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
# displaying the distance
def show(self):
    print("Feet= ",self.feet, "Inches= ",self.inches)
a,b= (input("Enter feet and inches of distance1: ")).split()
a,b =[int(a),int(b)]
c,d= (input("Enter feet and inches of distance2: ")).split()
c,d =[int(c),int(d)]
d1 = distance(a,b)
d2 = distance(c,d)
if(d1>d2):
    print("Distance1 is greater than Distance2")
else:
    print("Distance2 is greater or equal to Distance1")
d3=d1+d2
print("Sum of the two Distance is:")
d3.show()
```

Резултат:

Enter feet and inches of distance1: 8 4

Enter feet and inches of distance2: 6 9

Distance1 is greater than Distance2

Sum of the two Distance is:

Feet= 15 Inches= 1

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
main.py
1 class distance:
2     def __init__(self,f,i):
3         self.feet=f
4         self.inches=i
5 # overloading of binary operator > to compare two distances
6     def __gt__(self,d):
7         if(self.feet>d.feet):
8             return(True)
9         elif((self.feet==d.feet) and (self.inches>d.inches)):
10            return(True)
11        else:
12            return(False)
13 # overloading of binary operator + to add two distances
14    def __add__(self, d):
15        i=self.inches + d.inches
16        f=self.feet + d.feet
17        if(i>=12):
18            i=i-12
19            f=f+1
20        return distance(f,i)
21 # displaying the distance
22    def show(self):
23        print("Feet= ",self.feet, "Inches= ",self.inches)
24 a,b= (input("Enter feet and inches of distance1: ")).split()
25 a,b =[int(a),int(b)]
26 c,d= (input("Enter feet and inches of distance2: ")).split()
```

Shell

```
Enter feet and inches of distance1: 8 4
Enter feet and inches of distance2: 6 9
Distance1 is greater than Distance2
Sum of the two Distance is:
Feet= 15 Inches= 1
> |
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

METHOD OVERRIDING IN PYTHON

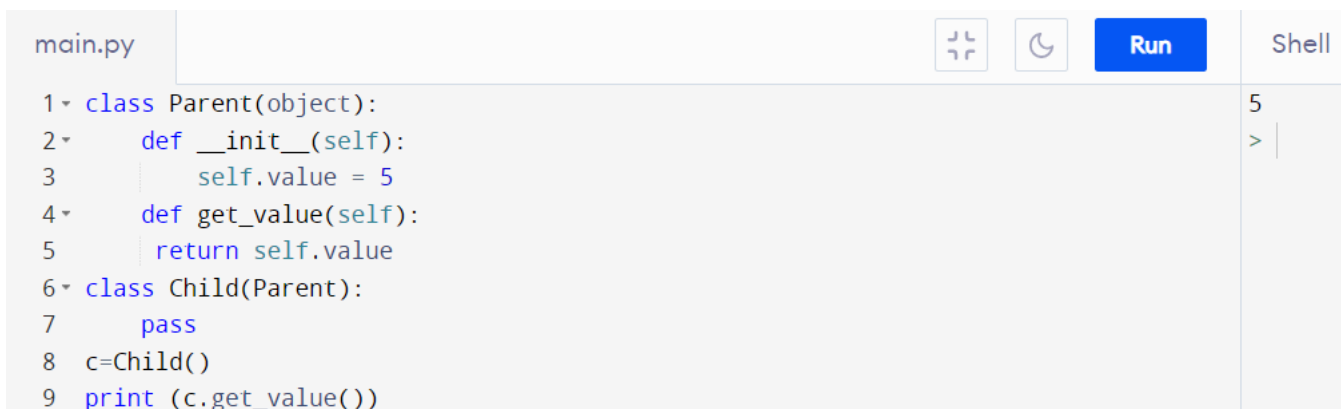
Отменянето е способността на клас да промени изпълнението на метод, предоставен от един от неговите предшественици. Отмяната е много важна част от ООП, тъй като това е функцията, която кара наследяването да използва пълната си мощ. Чрез отмяната на метода даден клас може да "копира" друг клас, избягвайки дублиране на код, и в същото време да подобри или персонализира част от него. Следователно отмяната на метода е строга част от механизма за наследяване.

Що се отнася до повечето ООП езици, **наследяването в Python работи чрез имплицитно делегиране**: когато обектът не може да удовлетвори заявка, той първо се опитва да препрати заявката до своите предшественици, следвайки специфичните езикови правила в случай на множествоно наследяване.

Пример:

Да се създаде клас Parent и наследник Child. В Parent да се създаде метод `__init__` (конструктор) и метод `get_value`. Child наследява метода `get_value`.

```
class Parent(object):
    def __init__(self):
        self.value = 5
    def get_value(self):
        return self.value
class Child(Parent):
    pass
c=Child()
print (c.get_value())
```



```
main.py
1 class Parent(object):
2     def __init__(self):
3         self.value = 5
4     def get_value(self):
5         return self.value
6 class Child(Parent):
7     pass
8 c=Child()
9 print (c.get_value())
```

5

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Както можете да видите, класът **Child** е празен, но тъй като се наследява от родителя, Python поема отговорността за маршрутизиране на всички извиквания на методи. Така че можете да използвате метода `get_value()` на Child обекти и всичко работи според очакванията.

Резултат:

5

PYTHON SPECIAL UNIT

Рамката за Unit тестване първоначално е вдъхновена от JUnit е и подобен на основните рамки за тестване на модули на други езици. Той поддържа автоматизация на тестовете, споделяне на код за настройка и изключване за тестове, агрегиране на тестове в колекции и независимост на тестовете от рамката за отчитане.

За да постигне това, Unittest поддържа някои важни концепции по ОО начин:

test fixture

test fixture представлява подготовката, необходима за извършване на един или повече тестове и всички свързани действия за почистване. Това може да включва например създаване на временни или прокси бази данни, директории или стартиране на сървърен процес.

test case

Тестовият случай е индивидуалната единица за тестване. Той проверява за специфичен отговор на определен набор от входове. Unittest предоставя основен клас, `TestCase`, който може да се използва за създаване на нови тестови случаи.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

test suite

test suite (Тестовият пакет) е колекция от тестови случаи, тестови пакети или и двете. Използва се за агрегиране на тестове, които трябва да се изпълняват заедно.

test runner

Тестовият елемент е компонент, който ръководи изпълнението на тестове и предоставя резултата на потребителя. Участникът може да използва графичен интерфейс, текстов интерфейс или да върне специална стойност, за да посочи резултатите от изпълнението на тестовете.

OBJECT REPRESENTATION

Destroying Objects (Garbage Collection) - Унищожаване на обекти (събиране на боклук)

Python изтрива ненужните обекти (вградени типове или екземпляри на клас) автоматично, за да освободи пространството в паметта. Процесът, чрез който Python периодично възстановява блокове памет, които вече не се използват, се нарича събиране на боклук.

Колекторът за боклук на Python работи по време на изпълнение на програмата и се задейства, когато броят на препратките на обект достигне нула. Референтният брой на обекта се променя, тъй като броят на псевдонимите, които сочат към него, се променя.

Броят на препратките на обект се увеличава, когато му се присвои ново име или се постави в контейнер (списък, кортеж или речник). Броят на препратките на обекта намалява, когато той бъде изтрит с `del`, като препратката му бъде преназначена или препратката му излезе извън обхвата. Когато референтният брой на обекта достигне нула, Python го събира автоматично.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
a = 40      # Create object <40>
b = a       # Increase ref. count of <40>
c = [b]     # Increase ref. count of <40>
```

```
del a       # Decrease ref. count of <40>
b = 100     # Decrease ref. count of <40>
c[0] = -1   # Decrease ref. count of <40>
```

Обикновено няма да забележите, когато събирачът за боклук унищожи неизползван екземпляр и възстанови мястото му. **Но един клас може да реализира специалния метод `del()`, наречен деструктор, който се извиква, когато екземплярът е на път да бъде унищожен. Този метод може да се използва за почистване на всички ресурси извън паметта, използвани от даден екземпляр.**

ATTRIBUTE BINDING

Attributes of class objects

Обикновено указвате атрибут на обект на клас чрез свързване на стойност към идентификатор в тялото на класа.

Пример:

```
class C1(object):
```

```
    x = 23
```

```
print(C1.x) # prints: 23
```

```
1 class C1(object):
2     x = 23
3     print(C1.x) # prints: 23
```

23
> |

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Обектът на класа C1 има атрибут с име x, обвързан със стойността 23, и C1.x се отнася до този атрибут. Можете също да обвързвате или развързвате атрибути на клас извън тялото на класа.

Пример:

```
class C2(object): pass
```

```
    C2.x = 23
```

```
print(C2.x)
```

```
# prints: 23
```

main.py	  	Shell
<pre>1 class C1(object): 2 x = 23 3 print(C1.x) # prints: 23 4 5 class C2(object): pass 6 C2.x = 23 7 print(C2.x) 8 # prints: 23</pre>		<pre>23 23 > </pre>

Вашата програма обикновено е по-четлива, ако свързвате и по този начин създавате атрибути на клас само с изрази в тялото на класа. Въпреки това, повторното им свързване на друго място може да е необходимо, ако искате да пренасяте информация за състоянието на ниво клас, а не инстанция; Python ви позволява да направите това, ако желаете. Няма разлика между атрибут на клас, създаден в тялото на класа, и такъв, създаден извън тялото чрез присвояване на атрибут. Както ще обсъдим скоро, всички екземпляри на класа споделят всички атрибути на класа. Инструкцията за клас имплицитно задава някои атрибути на класа. Атрибутът `__name__` е низът на идентификатора на името на класа, използван в израза за клас. Атрибута `__bases__` е наборът от обекти на класа, даден като базови класове в оператора за клас. Например, използвайки класа C1, който току-що създадохме:

```
print(C1.__name__, C1.__bases__)
```

```
# prints: C1 (<type 'object'>,)
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Класът също има атрибут `__dict__`, обектът за преобразуване, който класът използва за задържане на други атрибути (известен още като пространство от имена); в класовете това съпоставяне е само за четене. В изрази, които са директно в тялото на класа, препратките към атрибутите на класа трябва да използват просто име, а не напълно квалифицирано име. Например:

```
class C3(object):
```

```
    x = 23
```

```
    y = x + 22 # must use just x, not C3.x
```

Въпреки това, в изрази в методи, дефинирани в тялото на класа, препратките към атрибутите на класа трябва да използват напълно квалифицирано име, а не просто име. Например:

```
class C4(object): x = 23
```

```
    def amethod(self):
```

```
        print(C4.x) # must use C4.x or self.x, not just x!
```

Имайте предвид, че препратките към атрибути (т.е. израз като `C.s`) имат по-богата семантика от тези на свързването на атрибути.

Function definitions in a class body

Повечето тела на класове включват изрази `def`, тъй като функциите (известни като методи в този контекст) са важни атрибути за повечето обекти на класа. Изявлението `def` в тялото на клас се подчинява на правилата, представени във „Функции“. В допълнение, метод, дефиниран в тялото на класа, има задължителен първи параметър, условно наречен `self`, който се отнася до инстанцията, на която се извиква метода. Параметърът `self` играе специална роля при извикванията на методи. Ето пример за клас, който включва дефиниция на метод:

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

main.py			Run	Shell
<pre>1 class C1(object): 2 x = 23 3 print(C1.x) # prints: 23 4 5 class C2(object): pass 6 C2.x = 23 7 print(C2.x) 8 # prints: 23 9 0 class C3(object): 1 x = 23 2 y = x + 22 # must use just x, not C3.x 3 print(x, y) 4 5 class C4(object): 6 x = 23 7 def amethod(self): 8 print(C4.x) # must use C4.x or self.x, not just x!</pre>				<pre>23 23 23 45 > </pre>

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

```
class C5(object):  
    def hello(self):  
        print('Hello')
```

MEMORY MANAGEMENT IN PYTHON

Управлението на паметта в Python включва стек, съдържаща всички Python обекти и структури от данни. Управлението на стека се осигурява вътрешно от мениджъра на паметта на Python. Мениджърът на паметта на Python има различни компоненти, които се занимават с различни аспекти на динамично управление на съхранение, като споделяне, сегментиране, предварително разпределение или кеширане.

На най-ниското ниво разпределителят на необработена памет гарантира, че има достатъчно място в стека за съхранение на всички данни, свързани с Python, като взаимодейства с мениджъра на паметта на операционната система. В допълнение към разпределителя на необработената памет, няколко специфични за обекта разпределители работят върху една и съща част от паметта и прилагат различни политики за управление на паметта, адаптирани към особеностите на всеки тип обект.

Например, целочислените обекти се управляват по различен начин в рамките на стека, отколкото низовете, кортежите или речниците, тъй като целите числа предполагат различни изисквания за съхранение и компромиси между скорост/пространство. По този начин мениджърът на паметта на Python делегира част от работата на специфичните за обект разпределители, но гарантира, че последните работят в границите на heap-а.

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Важно е да се разбере, че управлението на стека на Python се извършва от самия интерпретатор и че потребителят няма контрол върху нея, дори ако редовно манипулира обектни указатели към блокове памет в стека. Разпределението на пространството на стека за обекти на Python и други вътрешни буфери се извършва при поискване от мениджъра на паметта на Python чрез функциите на API на Python/C. Освен това се предоставят следните набори от макроси за директно извикване на разпределителя на паметта на Python, без да се включват функциите на C API, изброени по-горе. Имайте предвид обаче, че използването им не запазва двоична съвместимост между версиите на Python и следователно е отхвърлено в модулите за разширение.

`PyMem_MALLOC(size)`

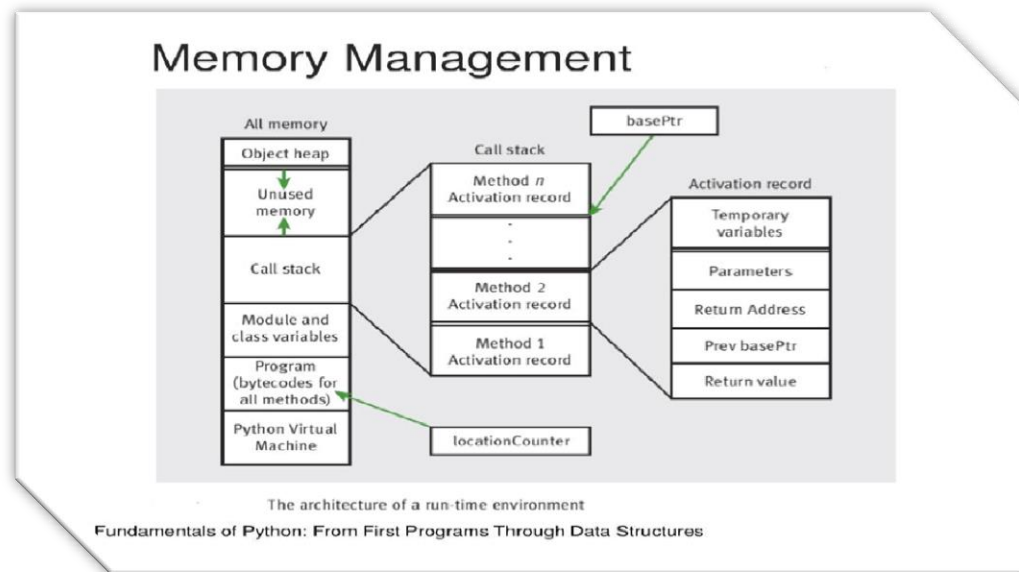
`PyMem_NEW(type, size)`

`PyMem_REALLOC(ptr, size)`

`PyMem_RESIZE(ptr, type, size)`

`PyMem_FREE(ptr)`

`PyMem_DEL(ptr)`



Фиг.2

Special properties of classes

Обектите на класа поддържат два вида операции: препратки към атрибути и инстанциране. Препратките към атрибути използват стандартния синтаксис, използван за всички препратки към атрибути в Python: `obj.name`. Валидни имена на атрибути са всички имена, които са били в пространството за имена на класа, когато е създаден обектът на класа. И така, ако дефиницията на класа изглеждаше така:

```
class MyClass:
```

```
    """A simple example class"""
```

```
    i = 12345
```

```
    def f(self):
```

```
        return 'hello world'
```

тогава `MyClass.i` и `MyClass.f` са валидни препратки към атрибути, връщащи съответно цяло число и обект на функция. Атрибутите на класа също могат да бъдат присвоени, така че можете да промените стойността на `MyClass.i` чрез присвояване. `doc` също е валиден атрибут, връщайки `docstring`, принадлежащ на класа: "Прост примерен клас". Инстанцирането на клас използва нотация на функция. В случая, обектът на класа е функция без параметри, която връща нов екземпляр на класа. Например (приемайки горния клас):

```
x = MyClass()
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

създава нов екземпляр на класа и присвоява този обект на локалната променлива `x`. Операцията за създаване на инстанция („извикване“ на обект от клас) създава празен обект. Много класове обичат да създават обекти с екземпляри, персонализирани към определено първоначално състояние. Следователно един клас може да дефинира специален метод с име `__init__()`, както следва:

```
def __init__(self):  
    self.data = []
```

Когато даден клас дефинира метод `__init__()`, инстанцирането на клас автоматично извиква `__init__()` за новосъздадения екземпляр на клас. Така че в този нов пример , инициализиран екземпляр може да бъде получен чрез:

```
x = MyClass()
```

Разбира се, методът `__init__()` може да има аргументи за по-голяма гъвкавост. В този случай аргументите, дадени на оператора за създаване на клас, се предават на `init ()`. Например,

```
>>> class Complex:  
...     def __init__(self, realpart, imagpart):  
...         self.r = realpart  
...         self.i = imagpart  
...  
>>> x = Complex(3.0, -4.5)  
>>> x.r, x.i (3.0, -4.5)
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

SLOTS AND PRIVATE ATTRIBUTES

Слотове

Слотовете осигуряват специален механизъм за намаляване на размера на обектите. Това е концепция за оптимизиране на паметта на обекти.

Синтаксис:

```
class myClass(object):  
    # defining the slots  
    __slots__ = (par1, par2)  
    def __init__(self, *args, **kwargs):  
        # initializing the values  
        self.par1 = value1  
        self.par2 = value2
```

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

Пример 1:

defining the class.

class gfg:

defining the slots.

__slots__ = ('course', 'price')

def __init__(self):

initializing the values

self.course = 'DSA Self Paced'

self.price = 3999

create an object of gfg class

a = gfg()

print the slot

print(a.__slots__)

print the slot variable

print(a.course, a.price)

main.py	Run	Shell
<pre>1 # defining the class. 2 class gfg: 3 # defining the slots. 4 __slots__ = ('course', 'price') 5 def __init__(self): 6 # initializing the values 7 self.course = 'DSA Self Paced' 8 self.price = 3999 9 # create an object of gfg class 10 a = gfg() 11 # print the slot 12 print(a.__slots__) 13 # print the slot variable 14 print(a.course, a.price)</pre>		<pre>('course', 'price') DSA Self Paced 3999 ></pre>

Резултат:

('course', 'price')

DSA Self Paced 3999

Тема 4_1. Обектно-ориентирано програмиране. Класове и обекти. Капсулиране, атрибути и свойства. Наследяване. Предефиниране на функционалността на базовия клас. Клас атрибути и статични методи. Низово представяне на обект

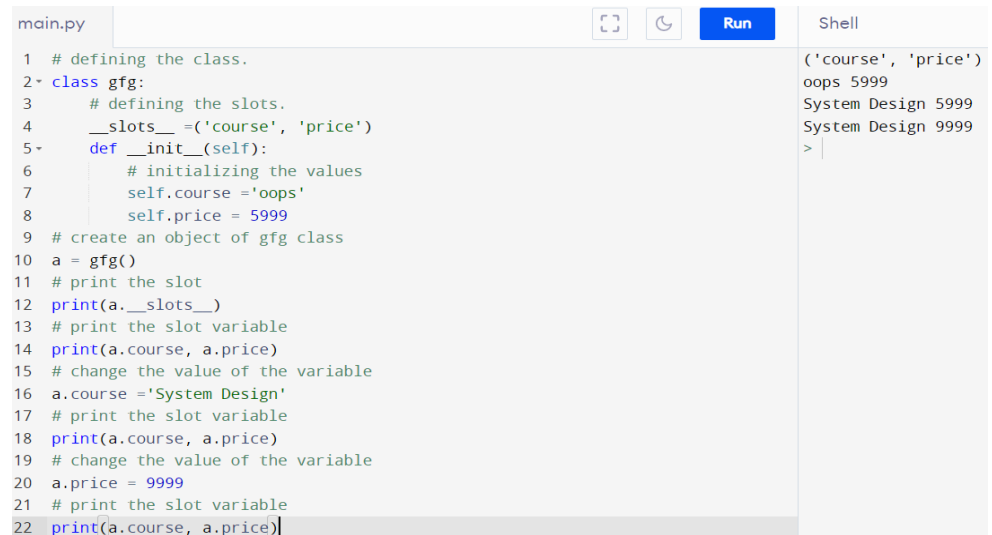
Пример 2:

```
# defining the class.
class gfg:
    # defining the slots.
    __slots__ = ('course', 'price')
    def __init__(self):
        # initializing the values
        self.course = 'oops'
        self.price = 5999
# create an object of gfg class
a = gfg()
# print the slot
print(a.__slots__)
# print the slot variable
print(a.course, a.price)
# change the value of the variable
a.course = 'System Design'
# print the slot variable
print(a.course, a.price)
```

```
# change the value of the variable
a.price = 9999
# print the slot variable
print(a.course, a.price)
```

Резултат:

```
('course', 'price')
oops 5999
System Design 5999
System Design 9999
```



```
main.py  Run Shell
1 # defining the class.
2 class gfg:
3     # defining the slots.
4     __slots__ = ('course', 'price')
5     def __init__(self):
6         # initializing the values
7         self.course = 'oops'
8         self.price = 5999
9 # create an object of gfg class
10 a = gfg()
11 # print the slot
12 print(a.__slots__)
13 # print the slot variable
14 print(a.course, a.price)
15 # change the value of the variable
16 a.course = 'System Design'
17 # print the slot variable
18 print(a.course, a.price)
19 # change the value of the variable
20 a.price = 9999
21 # print the slot variable
22 print(a.course, a.price)
```

```
('course', 'price')
oops 5999
System Design 5999
System Design 9999
> |
```

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009

COMPOSITION

Композицията означава, че един обект познава друг обект и изрично му делегира някои задачи. Докато наследяването е имплицитно, композицията е изрична: в Python обаче нещата са много по-интересни от това =).

Първо, нека приложим класическа композиция, която просто прави обект част от другия като атрибут.

Основната цел на композицията е да „отпусне“ връзката между обектите. Този малък пример показва, че сега SecurityDoor е обект и не вече е Door, което означава, че вътрешната структура на Door не се копира.

За този много прост пример и Door, и SecurityDoor не са големи класове, но в реална система обектите могат да бъдат много сложни; това означава, че тяхното разпределение консумира много памет и ако една система съдържа хиляди или милиони обекти, това може да бъде проблем.

ПРИМЕР